

國立中央大學
114 學年度 第一學期

軟體工程 期末書面報告

G AI 應用文件
寵物領養平台

組別：第十七組

組員：

114423001 陳柏甫

114423007 潘維晟

114423042 劉和媛

114423043 李嶧

目錄

1. 需求.....	3
a. spec.md (Use Case 草稿).....	3
b. Use case (初版).....	5
c. Use Case (根據程式修正).....	5
2. 設計.....	5
a. 邏輯設計 (Logical Design)	5
b. 細部設計 (Detailed Design).....	8
3. 程式.....	9
a. 程式生成.....	9
4. 測試.....	15
a. 單元測試.....	15
b. 整合測試.....	16
c. E2E 端對端測試.....	16

以下我們從軟體工程各階段去看我們如何應用 AI，包含需求分析、系統設計到程式生成以及測試。

1. 需求

a. spec.md (Use Case 草稿)

首先我們先利用 /speckit.specify 建立初步的需求規格文件。

在 prompt 中我們有先給他了 project、vision、mission、goals、target_users 和 key_features 等資訊。

關於 project 的相關資訊，是先透過 ChatGPT 整理小組的想法。

```
Follow instructions in speckit.specify.prompt.md.
project: 貓狗領養平台
vision: 讓更多人以領養代替購買的方式，幫助流浪找到新家，建立友善的動物送養環境。
mission:


- 提供一個透明、安全的線上領養與送養平台。
- 讓飼主能透過平台將無法繼續飼養的毛小孩送養給適合的新主人。
- 提供管理員有效率的後台管理功能。
- 提供即時客服機器人，回答與領養、照顧動物相關的問題。


goals:


- 會員能瀏覽、申請領養動物。
- 飼主能上架送養動物資料。
- 管理員能審核申請、更新與刪除動物資訊。
- 客服機器人能即時回答使用者問題。


target_users:


- 想領養貓狗的使用者。
- 想送養毛小孩的飼主。
- 負責平台維護與資料審核的管理員。


key_features:


- 使用者註冊與登入系統。
- 領養申請與審核流程。
- 動物資訊管理（新增、修改、刪除、瀏覽）。
- 即時客服機器人（領養與照護問答）。
- 送養刊登系統。

```

透過上面的 prompt 即可生成第一版的需求規格文件，包含 User Story、Requirements、Functional Requirements、Key Entities 等內容。

根據遇到的問題下指令去修改

例如：平台中腳色不夠明確：

```
Add Context...
目前感覺管理員的腳色不夠明確
Agent GPT-4o
```

Speckit 隨即調整 spec.md，新增腳色與權限摘要。

角色與權限摘要

以下角色與其主要能力為平台核心授權模型，請在實作時以 RBAC 與最小權限原則為準：

- 訪客 (Visitor)
 - 描述：未登入或未註冊的使用者。
 - 能力：可瀏覽平台上所有公開資訊（例如動物清單、照片、基本資料）；可使用搜尋與篩選。無法建立、修改或申請任何資料；若要進行領養或送養，需註冊為會員。
- 一般會員 (General Member)
 - 描述：已註冊的個人會員，可作為領養者或個人送養者。
 - 能力：
 - 建立與編輯個人資料、偏好設定。
 - 刊登個人送養資訊（含上傳照片與描述）；送養刊登在審核前可修改，通過審核後僅可檢視或下架。
 - 提出領養申請（系統禁止對自己刊登的動物提出申請）。
 - 在平台內接收通知並查看申請進度（Notification Center，MVP）。
- 收容所會員 (Shelter Member)
 - 描述：經平台審核通過的收容所帳號，由單一負責人或代理人操作（本版本不支援多成員管理）。
 - 能力：
 - 以機構身份批次刊登與管理所屬動物（CSV/JSON 匯入）。
 - 檢視並管理該收容所的領養申請、刊登狀態與報表（指派、批次狀態更新）。
 - 編輯、更新所屬收容所的動物資料與醫療紀錄（依權限與審核流程）。
- 管理員 (Admin)
 - 描述：平台管理與稽核人員，具最高權限。
 - 能力：
 - 審核會員與收容所註冊申請；審核動物刊登、醫療紀錄與申請。
 - 管理領養紀錄、處理檢舉、封鎖帳號、發布公告與變更平台設定。
 - 擁有新增、編輯、刪除任何資料的權限（僅限授權管理員等級）。
 - 所有重要操作應寫入 AuditLog 以供稽核。

注：在資料模型上建議使用單一 `role` 欄位（enum，如 GENERAL_MEMBER | SHELTER_MEMBER | ADMIN）；若未來需支援收容所內多位 staff，可再引入 `shelter_memberships` 或獨立的組織/成員表以擴充權限。

過程中也會下指令去檢查是否合理。

我們可以針對每一個 user story 去檢查是否合理嗎？

檢查文件一致性。

可以確認一下整份文件的一致性？

spec.md:1-535

檢查規格文件與資料庫內容是否相符。

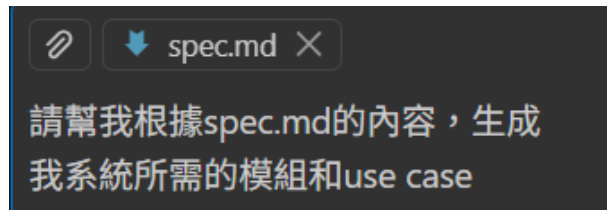
請問現在 erd.md 有符合 spec.md 的內容？

已檢查並部分對齊：erd.md 與 spec.md 的比較結果與建議（中文）

簡短回答：erd.md 大致上符合 spec.md 的內容；我做了幾處小的文件對齊（已在工作區中修改），但有幾個需要你確認或決策的地方（下方列出）。

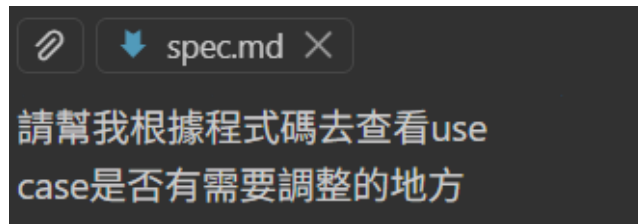
b. Use case (初版)

透過 Bmad 先生成初版 use case



c. Use Case (根據程式修正)

先請 AI 檢查，之後再透過人工去做 use case 的調整



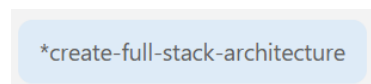
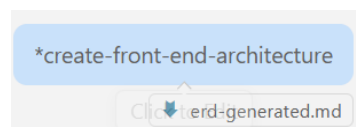
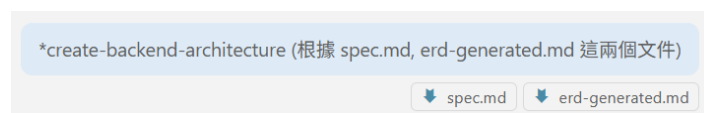
2. 設計

a. 邏輯設計 (Logical Design)

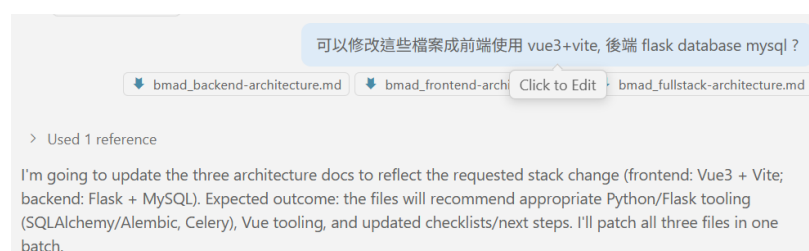
i. 系統架構圖 (初版)

使用 BMad method architect 的指令：

- *create-backend-architecture
- *create-front-end-architecture
- *create-full-stack-architecture



指定系統程式語言。



系統架構圖



ii. ERD (初版)

在生成 spec.md (Use Case 草稿)，有同時請 AI 產生 ERD 的出版內容。

同時利用調整過後的 spec.md 產生初版的資料庫(ERD)。



iii. 文件 B

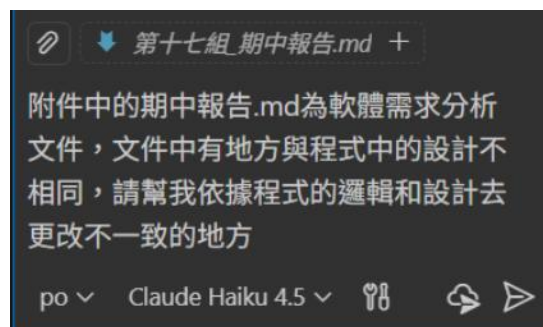
透過以下指令生成初版文件 B，並且再根據手動挑整確認需求。

我現在要生成系統需求規格分析書，我會給你三個檔案，第一個是老師規範的系統需求規格分析書範本(格式)，第二、三個檔案是我要提供給你生成系統需求規格分析書的資料，你可以做到嗎？



(程式開發完)

根據程式修改文件 B，並且並且再根據手動挑整確認需求是否符合預期。

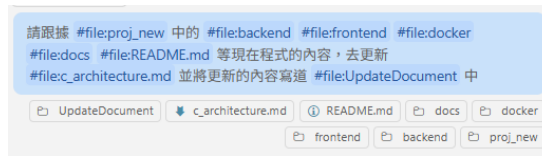


iv. 文件 C (根據程式修正)

1. 利用正確的程式對齊回系統文件

包含系統架構圖以及 ERD 等等，屬於文件 C 的內容。

這邊使用 BMad Architect Agent。



2. 系統架構圖(mvc)



修正 MVC 中，data layer 應該是由統一介面連到資料庫，下面是與 AI 的對答。

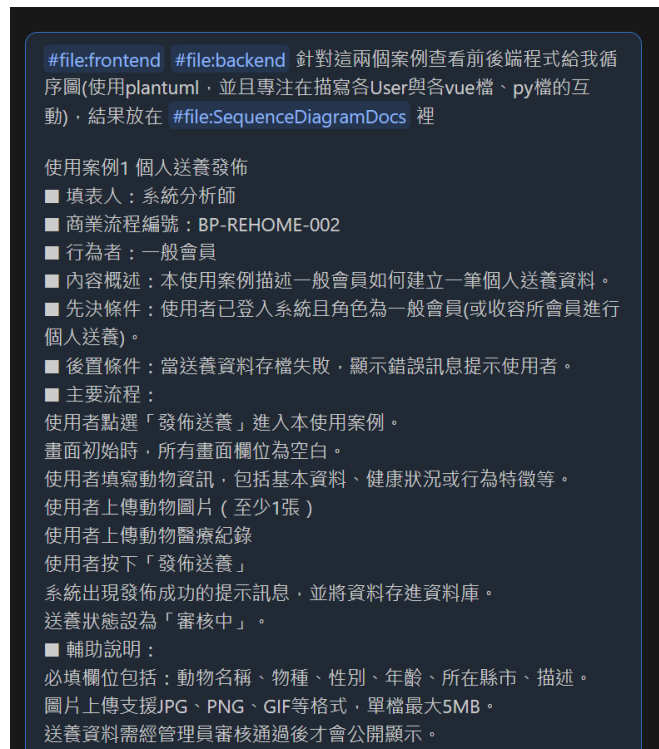


b. 細部設計 (Detailed Design)

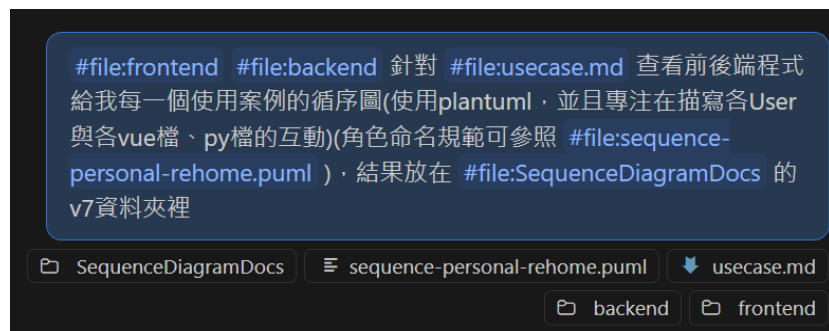
i. 循序圖

使用 BMad method architect 進行生成

先針對兩個使用案例生成循序圖查看成效



再將生成的循序圖進行手動調整修改後，進行其他 usecase 的循序圖生成並可作為參考。



3. 程式

a. 程式生成

使用 BMad method dev 進行生成

i. 專案啟動與程式碼生成

1. 初始搭建

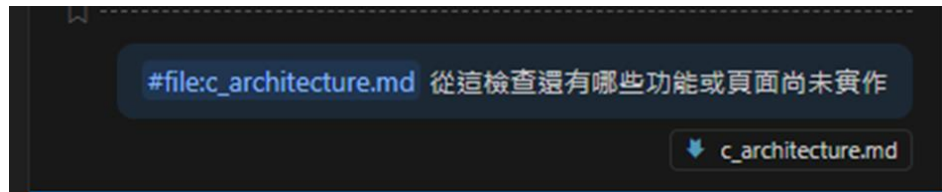
根據 B、C、ERD、MVC 等規格文件要求 Copilot
生成初始的前後端應用程式骨架



ii. 需求確認與功能實作

1. 需求對焦

反覆參照 C 文件「架構設計文件」，檢查並確認尚未實作的功能，利用 AI 協助判斷未成功能的實作優先級



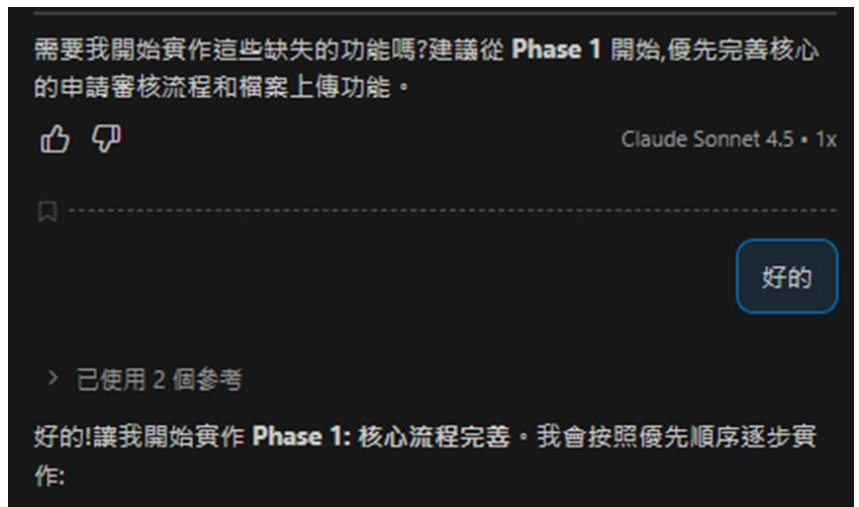
2. 實作規劃

請求 Copilot 提供更詳細的實作規劃(補足前面文件的不足)策略性地先完成 Quick Win(快速有成效)的功能



3. 功能開發

依照 AI 的建議進行分階段、分模組進行系統實作實作具體功能



iii. 系統除錯與問題修復

1. 初步檢查

檢查 AI 生成程式碼的初步錯誤



2. 解決重點功能錯誤

如送養表單編輯、用戶登入等問題解決



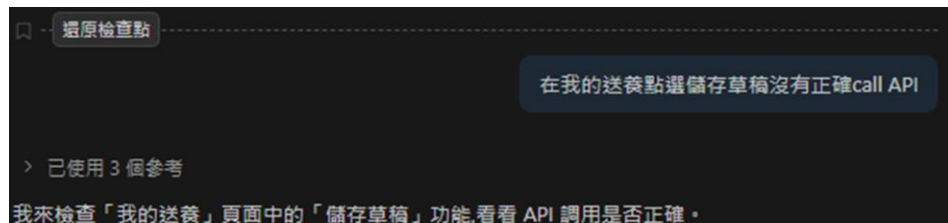
3. 前端顯示與邏輯

頁面加載錯誤、前端操作邏輯、網頁路由等問題解決



4. 後端與 API

call API 錯誤、後端執行錯誤解決



5. 未知錯誤處理

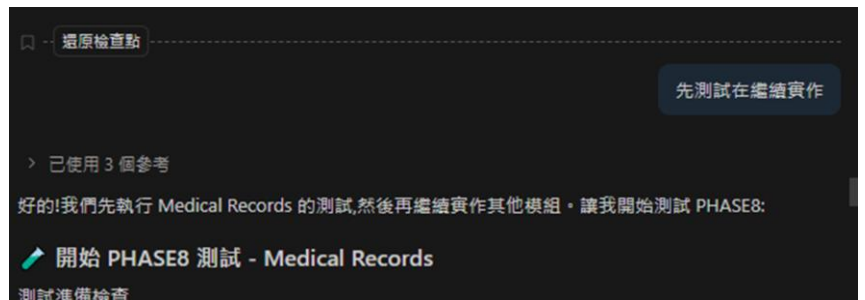
大家最熟知的直接貼 log...



iv. 測試與驗證

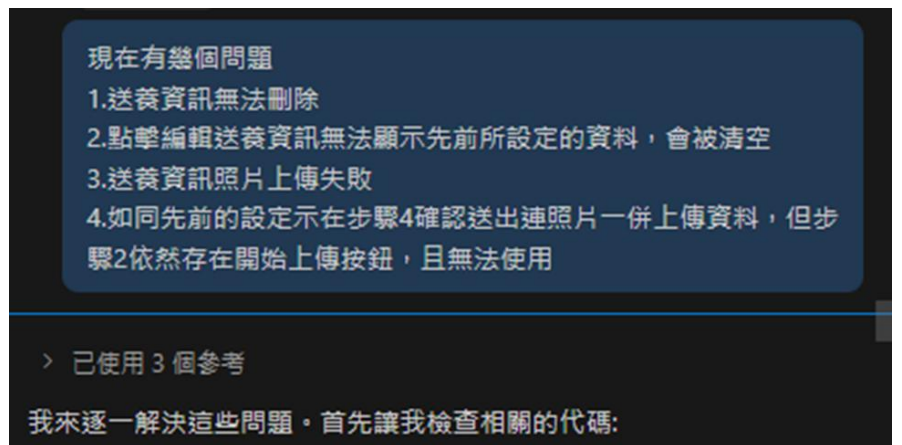
1. 開發中 API 端點測試

在實作過程中，穿插進行功能測試



2. 開發中 API 端點測試

進行實際手動在前端測試，並要求 AI 根據測試結果補足系統缺少的功能或修正問題



v. 重構、清理與優化

1. 程式精簡

依據 C 文件架構，清除前後端冗餘的程式碼、改寫冗長的程式



2. 功能優化

針對現有操作功能不友善部分進行優化



3. 程式重構

確保 API 需依照業務邏輯、API 端點分離



vi. 環境與部署

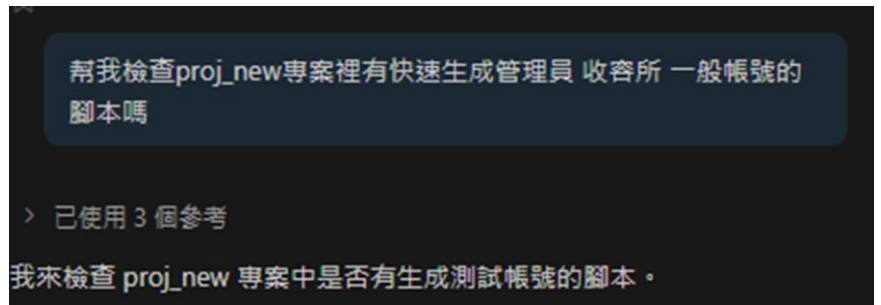
1. 資料庫

生成資料庫遷移 (Migration) 檔案，方便快速建立
資料庫結構



2. 腳本工具

生成快速建立測試帳號的腳本



vii. 文件管理

1. 帳號文件

更新系統可登入的帳號密碼及身份文件



2. 專案文件

依據當前程式碼更新、整理開發及說明文件



viii. 泛用程式詢問

1. 程式詢問

針對特定的程式碼或概念問題向 Copilot 提問



4. 測試

a. 單元測試

(使用 BMad-method 的 qa 進行，使用模型為 Claude Sonnet 4) 後來有重新調整，單元測試的內容。原本期末簡報是按照使用案例，但後來改成讓 AI 去看程式碼直接生單元測試，因為這邊想要直接使用 pytest 的覆蓋率，並且聚焦於每一個 service 的業務邏輯。另外，也有給 AI 關於測試的上課講義，讓他可以生出正確的測試。

大致與 Agent 的討論過程如下：

我了解一下 還是說我的單元測試不應該依照 使用案例去寫 而是真的程式碼去寫嗎？

目前測試策略的分析

你的測試檔案命名：

- test_animals_detail_service_useCase1_3.py → 按使用案例命名
- test_application_submit_integration_useCase1_4.py → 按使用案例命名

但測試內容：

- 單元測試：測試 `AnimalService.getAnimal()` 這個方法
- 整合測試：測試完整的 API endpoint 流程

正確的測試組織方式

單元測試 (Unit Tests) → 應該按照程式碼結構

目的：測試每個 class/method 的邏輯正確性

```
# 推薦命名：按照程式碼結構
tests/unit/
├── test_animal_service.py      # 測試 animal_service.py 所有方法
│   ├── TestAnimalServiceGetAnimal
│   ├── TestAnimalServiceListAnimals
│   ├── TestAnimalServiceSearchAnimals
│   └── TestAnimalServiceCreateAnimal
├── test_application_service.py # 測試 application_service.py
├── test_user_service.py
└── test_shelter_service.py
```

為什麼？

- 程式碼重複時容易維護 (改 service 就改對應的 test file)
- 容易計算覆蓋率 (每個 service 都有對應測試)
- 符合單元測試「測試最小單位」的原則

可以了解一下現在的進度？

> Used 1 reference
✓ Summarized progress and identified issues

目前測試進度總結

✓ 已完成的服務測試 (7個服務, 100% 通過率)

服務	測試數量	通過率	覆蓋率
animal_service	60	100%	95%
admin_service	23	100%	97%
application_service	39	100%	96%
audit_service	13	100%	100%
attachment_service	11	100%	100%
job_service	17	100%	90%
permission_service	20	100%	88%
小計	183	100%	95.1%

✖ 進行中的服務測試 (3個服務, 部分通過)

服務	測試數量	通過/總數	通過率	主要問題
medical_record_service	13	4/13	31%	Enum 值錯誤 (UserRole.REGULAR → GENERAL_MEMBER)
notification_service	15	9/15	60%	需要修正 Shelter 模組的 patch
user_service	14	9/14	64%	Enum 值錯誤 (同上)
小計	42	22/42	52%	-

✖ 尚未開始的服務測試 (2個服務)

- shelter_service - 預估需要 15-20 個測試
- email_service - 預估需要 8-10 個測試

b. 整合測試

(使用 BMad-method 的 qa 進行，使用模型為 Claude Sonnet 4)
大致過程與單元測試相通。

可以跟我說一下整合測試 設計的邏輯，以及為甚麼做這些測試。

> Used 1 reference
✓ Explained integration testing design principles

整合測試設計邏輯與目的

為什麼需要整合測試？

整合測試與單元測試的互補關係：

測試類型	單元測試	整合測試
測試目標	驗證 Service 層業務邏輯	驗證完整的 API 流程
測試範圍	單一方法 (隔離)	Route → Service → ORM → Database
依賴處理	Mock 所有外部依賴	真實的資料庫、ORM
發現問題	邏輯錯誤、邊界條件	組件整合問題、SQL 錯誤、權限問題

核心理由：

- 驗證組件協作 - Service 單獨正確，不代表與 Route、ORM、Database 整合後正確
- 真實環境模擬 - 測試真實的 HTTP 請求/響應流程
- 發現整合問題 - SQL 語法錯誤、外鍵約束、權限檢查等問題
- 契約測試 - 確保 API 契約 (請求/響應格式) 正確

測試覆蓋優先級

優先級 1: 核心功能 (必須測試)

範例：

- ✓ 用戶註冊、登入
- ✓ 動物列表瀏覽
- ✓ 領養申請提交
- ✓ 申請審核流程

理由：這些是系統最基本、最常用的功能

優先級 2: 權限控制 (必須測試)

範例：

- ✓ 僅擁有者可以編輯/刪除
- ✓ 僅管理員可以審核
- ✓ 未登入無法操作

理由：安全性關鍵，必須確保正確

優先級 3: 資料完整性 (重要)

範例：

- ✓ 重複申請檢查
- ✓ 外鍵約束
- ✓ 必填欄位驗證

理由：保護資料庫一致性

優先級 4: 邊界條件 (重要)

範例：

- ✓ 不存在的資源 (404)
- ✓ 無效的輸入 (400)

c. E2E 端對端測試

(使用 BMad-method 的 qa 進行，使用 Claude Sonnet 4)
AI 的角色：將需求/流程描述轉成可執行的測試腳本與資料，持續協助設計、錯誤歸因與改進建議。
使用原則：以 Playwright 驗證為核心執行器；AI 參與到「生成、優化、修復、總結」四個環節，而不是取代測試框架。
大致與 Agent 的討論過程如下：
將需求轉成測試：

輸入：以自然語言描述業務流程與約束（權限、異常、資料規則）。

AI 輸出：測試案例清單（正/反向路徑、邊界條件）、對應頁面/元件的 POM skeleton。

接下來再以相同方法修正測試腳本，直到達成目標。

請依據此流程『收容所上傳 → 管理員批准 → 用戶領養 → 收容所審核 → 用戶查看』，輸出：1) 正/反向路徑 2) 對應頁面/POM 動作 3) 斷言清單（UI 與 API） 4) 關鍵資料欄位與邊界。

SYSTEM_ARCHITECTURE.puml:1-36